# Edge Vision Transformer Hardware Acceleration

*From First Principles to Embedded Silicon*

## 1. Why This Problem Matters

Picture a small drone flying fast — 8 to 15 meters per second, roughly running speed on a bike — through an obstacle course of wires, rebar, and scaffolding. Its camera takes a picture 30 times a second, and every single picture, the drone's brain has about 33 milliseconds (a thirtieth of a second) to decide: keep flying straight, or dodge?

Vision Transformers (ViTs) are a newer, smarter kind of AI vision model — better than older CNN models at noticing things far apart in an image relate to each other (like the top and bottom of a long wire). The catch: they need more memory bandwidth than these tiny drones can spare.

### The stakes are lopsided, not symmetric:

- **Miss a real wire (false negative):** the drone hits it. Crash, drone destroyed, mission over.
- **Imagine a wire that isn't there (false positive):** the drone flinches and dodges nothing. Slightly wastes battery. No real harm.

So the safe design choice is: when in doubt, err toward caution — never sacrifice detection accuracy just to save power.

### Why ViTs strain small chips specifically

A regular CNN reuses the same small filter over and over across an image, so it does a lot of math per byte of data it reads — efficient. A ViT's “attention” step instead builds a giant table comparing every part of the image to every other part. For a 14×14 grid of image patches (196 patches total), that's a 196×196 table that has to be written to memory, read back for a math step (softmax), written again, then read once more. All that back-and-forth to memory is the actual bottleneck — not the raw amount of math. On a small chip like the Xilinx Zynq-7000 (a common low-cost FPGA board), that memory traffic alone can blow through the 5-watt power budget.

**In short:** ViTs see more of the picture at once, which is great for spotting hazards — but the “comparing everything to everything” step floods the memory bus on tiny hardware, and the whole system has to solve that without ever missing a real hazard.

**Quick Check**

**Q1. Why is a missed hazard (false negative) worse than a false alarm (false positive) for this drone?**

**A:** A missed hazard means the drone collides with a real obstacle at speed — a catastrophic, unrecoverable loss. A false alarm just costs a brief, safe detour and a little battery.

**Q2. Why do Vision Transformers strain small chips more than CNNs do?**

**A:** A CNN reuses the same small filter efficiently across the image. A ViT's attention step builds a full N×N comparison table that must be written to and read from memory repeatedly, which floods the memory bus rather than the math units.

**Q3. What is the drone's per-frame time budget, and why does it matter?**

**A:** About 33 milliseconds (for 30 FPS). Every decision — fly straight or dodge — must complete inside that window, so the hardware has to be fast and predictable, not just accurate.

## 2. The Three Tricks That Make It Work

### Trick 1: Shrink the numbers (Quantization)

Normal AI math uses 32-bit numbers (huge precision, huge cost). This design uses much smaller numbers instead:

- **Weights** (the model's learned parameters): squeezed down to 4 bits each — two of them fit in a single byte.
- **Activations** (the live data flowing through, i.e. the actual image signal): kept at 8 bits, a bit roomier.

Why the imbalance? Because subtle real-world signals — a thin wire barely visible against the sky — are exactly the kind of detail that gets erased if you round too aggressively. Keeping activations at 8 bits protects against that; squeezing only the weights to 4 bits still cuts memory traffic in half without risking a missed hazard. This mapping from a real number into a small integer range with a scale factor is the standard “affine quantization” approach used throughout embedded deep learning.

### Trick 2: Never build the giant table (Online/Streaming Softmax)

Instead of writing the full 196×196 comparison table to memory and reading it all back for normalization, the hardware processes scores one at a time as they stream in, updating a running “highest score so far” and a running “total” as it goes — the same trick used by FlashAttention, adapted here for a small FPGA chip instead of a GPU.

Worked example — three incoming scores, $x_1 = 4$, $x_2 = 6$, $x_3 = 3$:

| Step | New score | Running max | Running total |
|---|---|---|---|
| 1 | 4 | 4 | 1.0 |
| 2 | 6 | 6 | 1.25 |
| 3 | 3 | 6 | 1.375 |

Cross-check against the “do it the slow way” answer: 2^(4–6) + 2^(6–6) + 2^(3–6) = 0.25 + 1.0 + 0.125 = **1.375**. The streaming shortcut gives the identical answer without ever storing the full table.

## Trick 3: Avoid expensive math circuits (Base-2 exponentials)

Computing “e to the power of x” in hardware needs a costly circuit. Since computers already think in powers of 2, the design rewrites every exponential as a power of 2 instead (a fixed, one-time conversion factor of ~1.4427 handles the difference between “powers of e” and “powers of 2”). Splitting a power of 2 into a whole-number part and a fractional part means:

- The whole-number part becomes a *free* bit-shift (like multiplying by 2 just by moving digits over) — this uses zero of the chip's precious multiplier circuits.
- The fractional part is looked up in a small 64-entry table.

This is a well-established FPGA technique for avoiding costly floating-point exponentiation hardware.

### Quick Check

**Q1. Why are activations kept at 8 bits while weights are shrunk to 4 bits?**

**A:** Activations carry the live image signal, and subtle details (like a thin wire) can be erased by aggressive rounding. Weights can tolerate more compression, so squeezing only them still halves memory traffic without risking a missed hazard.

**Q2. What problem does the streaming (“online”) softmax avoid?**

**A:** It avoids ever writing the full N×N attention score table to memory. By tracking a running maximum and running total as scores stream in, the hardware gets the exact same answer one value at a time.

**Q3. Why use powers of 2 instead of natural exponentials (e^x) in hardware?**

**A:** Powers of 2 split into a whole-number part, computed as a free bit-shift, and a fractional part, looked up in a small table — avoiding the expensive floating-point exponentiation circuits that computing e^x directly would require.

## 3. How Fast Is It, Really?

**Question:** how long does it take to load a 64×64 weight matrix over a 32-bit data bus at 100 MHz, in packed 4-bit format vs. plain 8-bit format?

- Total weights: 64 × 64 = 4,096
- **4-bit (packed) version:** 4,096 × 4 bits = 16,384 bits = 2,048 bytes. Bus moves 4 bytes/cycle → 2,048 ÷ 4 = **512 cycles** → at 100 MHz (10 nanoseconds/cycle) = **5.12 microseconds**.
- **8-bit version:** 4,096 bytes ÷ 4 bytes/cycle = **1,024 cycles** = **10.24 microseconds**.

**Packing weights into 4 bits cuts loading time — and the energy spent moving data — exactly in half.** This is the practical payoff of Trick 1 above, expressed as a real number instead of an abstract claim.

**Quick Check**

**Q1. How many AXI-Stream cycles does it take to load a 64×64 INT4 weight matrix over a 32-bit bus?**

**A:** 512 cycles — 2,048 total bytes of packed 4-bit data, divided by 4 bytes moved per cycle.

**Q2. How much faster is the packed INT4 transfer compared to plain INT8?**

**A:** Exactly twice as fast: 512 cycles (5.12 µs) for INT4 versus 1,024 cycles (10.24 µs) for INT8.

**Q3. What does this cycle-count difference translate to in practice?**

**A:** Half the bus transactions means half the DRAM/AXI traffic — which directly translates into lower energy spent moving data and more thermal headroom inside the 5 W budget.

## 4. What Other Researchers Have Already Tried

| Project | What it did | Where it falls short |
|---|---|---|
| **FlashAttention** (Dao et al., 2022) | Invented the streaming-softmax trick this proposal reuses | Built for big GPUs with huge fast memory — doesn't say how to fit it onto a tiny FPGA's fixed-function circuits |
| **FlexViT** (Dymarkowski, Fu, Saha, Haris, & Cano, 2026) | Ran a full ViT on a similar cheap board (PYNQ-Z2) using 8-bit math everywhere | Used almost all the chip's fast on-board memory (83.9%) — confirms the memory problem is real and severe, but doesn't solve it with smaller-than-8-bit numbers |
| **SystolicAttention** (Lin et al., 2025) | Built the entire attention math, softmax included, directly into a grid of processing cells | Needs a bigger, pricier chip than the low-cost one this proposal targets |

**The gap this proposal fills:** nobody has combined (a) 4-bit weight packing, (b) the streaming softmax trick, and (c) the free bit-shift exponential trick into one design built specifically for a $100-class FPGA board with a 5-watt power ceiling. That's the genuinely open slot in the literature.

## Quick Check

**Q1. What does FlashAttention contribute, and what's its limitation here?**

**A:** It contributes the streaming-softmax math that avoids storing the full attention table. Its limitation is that it was designed for GPUs with large fast memory, not for a tiny FPGA's fixed-function circuits.

**Q2. What did FlexViT demonstrate about memory pressure on similar hardware?**

**A:** Running a full ViT on a comparable low-cost board (PYNQ-Z2) used 83.9% of the chip's on-board fast memory even at 8-bit precision — confirming the memory bottleneck is real and severe.

**Q3. What gap does this proposal fill that prior work hasn't?**

**A:** No existing design combines 4-bit weight packing, the streaming softmax trick, and the free bit-shift exponential trick together, specifically targeted at a $100-class, 5-watt FPGA board.

## 5. 12-Week Feasibility & Implementation Plan

A semester project succeeds or fails based on realistic milestones. Rather than attempting to synthesize a massive multi-layer model all at once, this plan partitions the workload: the ARM CPU handles tokenization, while the custom hardware focuses strictly on the memory-choked attention core.

| Weeks | Phase | Milestones & Deliverables |
|---|---|---|
| 1–3 | Software Model & Golden Vectors | • Set up a PyTorch reference model of a lightweight patch attention block.<br>• Quantize weights to INT4 and activations to INT8; verify thin-obstacle recall drops by less than 1%.<br>• Write a bit-accurate Python script modeling the Base-2 streaming softmax.<br>• Export binary test vectors (inputs.bin, weights.bin, golden_outputs.bin). |
| 4–7 | RTL Datapath Design (Verilog) | • Write synthesizable SystemVerilog for the 4-bit unpacker (splits 1 byte into two 4-bit signed numbers).<br>• Build the pipelined MAC compute array mapped cleanly to Xilinx DSP48E1 slices.<br>• Build the fused Base-2 streaming softmax unit using barrel shifters and a 64-entry LUT.<br>• Verify against golden vectors in Vivado xsim/Verilator until 100% bit-accurate. |
| 8–9 | SoC & AXI DMA Integration | • Package the RTL as an IP block with AXI4-Stream data ports and AXI4-Lite control registers.<br>• Build the Vivado Block Design connecting the Zynq ARM Processing System (PS) to the core via AXI DMA.<br>• Close timing on the Zynq XC7Z020 FPGA at ≥ 100 MHz. |
| 10–12 | Physical Bring-Up & Benchmarking | • Boot Linux on the PYNQ-Z2 board and write a C/Python DMA driver to push real image patches into the FPGA.<br>• Measure clock-cycle execution time using ARM PMU hardware counters; verify zero-jitter completion well within the 33 ms deadline.<br>• Measure power draw and verify that AXI bus traffic is halved.<br>• Assemble the final demo, code repository, and report. |

### What “Done” Looks Like at Week 12

- **Hardware Parity:** the custom IP running on the physical PYNQ-Z2 board matches the software golden vectors bit-for-bit.
- **Sub-33 ms Frame Deadline:** the attention core finishes computing its patch block in a fixed, predictable cycle count, guaranteeing ≥ 30 FPS flight-controller execution without latency spikes.
- **50% Memory Bus Reduction:** hardware counters confirm that streaming packed 4-bit weights cuts AXI bus transactions exactly in half compared to 8-bit baselines.
- **Under 5 Watts:** the full chip operates comfortably within the drone's strict 5 W thermal envelope.

## Quick Check

**Q1. Why is the workload split between the ARM CPU and the custom hardware, instead of building everything in RTL?**

**A:** The ARM CPU handles general tasks like tokenization, while the custom hardware focuses only on the true bottleneck — the attention core — which keeps the 12-week project realistic on limited chip resources.

**Q2. What must be verified before RTL design begins in Weeks 4–7?**

**A:** A software (PyTorch) reference model must be quantized to INT4/INT8 and shown to lose less than 1% thin-obstacle recall, with bit-accurate golden test vectors exported for later hardware verification.

**Q3. What are the four criteria that define “done” at Week 12?**

**A:** Bit-for-bit hardware/software parity, a fixed cycle count safely under the 33 ms frame deadline, a confirmed 50% cut in AXI memory-bus traffic, and total power draw under 5 watts.

# 6. Reflection: LLM Collaboration & Critical Review

## What the LLM got wrong, and how it was caught

During initial ideation, the LLM proposed implementing a full, end-to-end Vision Transformer (including LayerNorm, GeLU, multi-head attention, and multi-layer perceptron feed-forward blocks) entirely in RTL on an entry-level FPGA within 12 weeks. This was caught by checking the physical resource limits of the target chip: a standard Zynq-7000 XC7Z020 has only 220 DSP48E1 slices and 4.9 Mb of BRAM. Attempting to map both non-linear transcendental units and GEMMs for an entire network would exceed available logic slices or stall on place-and-route timing closure. The design was instead split into a heterogeneous system partition: the ARM Cortex-A9 host executes general network layers, while the custom PL accelerator focuses strictly on the true computational bottleneck — the memory-bound Multi-Head Attention score pipeline.

The LLM also initially suggested calculating standard natural exponentials (e^x) directly in fixed-point logic without specifying lookup table sizes. This hardware-blind suggestion was replaced with the Base-2 approximation (2^y), which splits the operation into a zero-DSP bit-shift and a 64-entry distributed LUT.

## What prompting approaches worked best

- **Constraint-First Prompts:** establishing strict physical boundaries up front (e.g., “Target platform is XC7Z020, 100 MHz clock, 32-bit AXI-Stream, 5W thermal budget, 12 weeks”) prevented the LLM from suggesting impractical data-center GPU frameworks or unrealistic tape-out architectures.
- **Adversarial Interrogation:** prompting the model to act as an unsparing architectural reviewer and challenge feasibility (e.g., “Where will this datapath stall? What makes this a toy project?”) exposed subtle bottlenecks like AXI DMA transfer overhead and dynamic activation outlier degradation.
- **Step-by-Step Derivations:** requiring the LLM to write out mathematical traces (such as the cycle-by-cycle Online Softmax recurrence) ensured that mathematical claims were verified before writing the text.

## What was learned that the LLM couldn't have provided directly

The LLM could explain the theoretical equations of FlashAttention and quantization, but it could not assess the operational stakes of the deployment environment. It required engineering judgment to realize that within an autonomous UAV flight system, the cost of errors is heavily asymmetric. The LLM treated false positives and false negatives as mathematically equal terms in a standard cross-entropy loss function. Analyzing the operational scenario made it clear that while false positives cause brief, recoverable flight detours, a false negative leads to a catastrophic collision with unspotted obstacles. This insight directly dictated the technical decision to maintain INT8 activations rather than aggressively quantizing everything to INT4.

## Project ideas for later phases

- **The Packed INT4/INT8 Streaming ViT Attention Engine (Selected):** implementing the custom SystemVerilog datapath featuring the on-the-fly sub-byte unpacker, DSP48 dual-MAC array, and fused Base-2 online normalizer, controlled via AXI DMA on a physical PYNQ-Z2 board.

- **Microarchitectural Side-Channel Mitigations for RISC-V Cryptographic Engines:** exploring cache-timing leakage on open-source RISC-V soft cores (such as CV32E40P) during post-quantum polynomial multiplication (ML-KEM/Kyber), designing an RTL-level randomized cache controller mitigation to prevent secret-key leakage.

## Quick Check

**Q1. What mistake did the LLM make about hardware resource limits, and how was it caught?**

**A:** It proposed building an entire transformer in RTL in 12 weeks. Checking the XC7Z020's real limits — only 220 DSP48E1 slices and 4.9 Mb of BRAM — showed this would overflow the chip, leading to a split design where the ARM CPU handles most layers and custom hardware handles only the attention bottleneck.

**Q2. Why did the false-positive/false-negative asymmetry change the quantization plan?**

**A:** Because a missed hazard (false negative) is catastrophic while a false alarm is merely inconvenient, the design keeps activations at a safer 8 bits instead of aggressively quantizing everything to 4 bits, protecting against dropped hazard signals.

**Q3. What are the two follow-on project ideas identified for later phases?**

**A:** (1) The Packed INT4/INT8 Streaming ViT Attention Engine on a physical PYNQ-Z2 board — the selected project — and (2) microarchitectural side-channel mitigations for RISC-V cryptographic engines against cache-timing leakage.

## 7. Reference List

**1.** Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 35, 16344–16359. arXiv:2205.14135.

**2.** Dymarkowski, H., Fu, X., Saha, R., Haris, J., & Cano, J. (2026). *FlexViT: A Flexible FPGA-based Accelerator for Edge Vision Transformers.* arXiv:2606.31938.

**3.** Demirkiran, C., Nair, L., Bunandar, D., & Joshi, A. (2024). *A blueprint for precise and fault-tolerant analog neural networks.* Nature Communications, 15, Article 5098.

**4.** Micikevicius, P., et al. (2022). *FP8 Formats for Deep Learning.* arXiv:2209.05433.

**5.** AMD/Xilinx Corporation. (2021). *Zynq-7000 SoC Technical Reference Manual* (UG585, v1.13).

**6.** Lin, J., Li, Y., Chen, G., & Bourgeat, T. (2025). *SystolicAttention: Fusing FlashAttention within a Single Systolic Array.* arXiv:2507.11331.